EXPERT-DRIVEN DEVELOPMENT

CAPSTONE

Full-Stack AI-Assisted Development

Direct AI to build, run, and deploy a complete web application — backend, frontend, data, chat, auth, integrations, container — in a single day.

Course 6
The Capstone

8 Hours
10 modules + assessment

Bonus / Elective
Advanced Builders

6 WEEK SIX OF SIX

Open in a second tab before you start — one click per “switch to editor” cue.

▸ **Slide-by-slide prompt hand-out**

SPEAKER NOTES

OPEN

Welcome to Week 6 — the capstone. Six weeks ago you took AI Fluency. Today you finish the program by directing AI to build a complete full-stack application from zero. Take a beat. This is not theory.

SET THE ROOM

- Confirm everyone can see your screen at full size in Teams.
- Tell them: "We will switch back and forth between this deck and a live editor a lot today. The deck is the map. The editor is the ground."
- Say out loud: "Speaker notes are on every slide if you're following along on the student page."

TONE

Calm, confident, slightly ceremonial — this is the capstone of the program. Avoid being precious; treat it as the natural next step from Course 4.

TRANSITION → "BEFORE WE START, LET'S DO A QUICK RECAP OF HOW WE GOT HERE."

How we got here

The six-week arc that ends today.

- **Week 1 — AI Fluency.** Six 201-level skills. The 80% quit problem. Centaur and Cyborg.
- **Week 2 — Builder Orientation.** First real prototype. Decomposition under fire.
- **Week 3 — Platform Training.** Three deployed Power Platform tools.
- **Week 4 — Advanced Workshop.** You mapped your frontier. You taught someone else.
- **Week 5 — Supervisor Orientation.** Leaders learned to evaluate proposals and protect the apprentice pipeline.
- **Week 6 — Today.** The frontier moves out from under Power Platform. You build the whole stack.

SPEAKER NOTES

WHAT TO SAY

Run this fast. The room has lived through five weeks of training; you are reminding them of the runway, not re-teaching it.

ANCHOR EACH WEEK IN ONE SENTENCE

- Week 1 is the foundation — the management framing, not the prompt tricks.
- Weeks 2–3 made everyone a builder inside the Power Platform envelope.
- Week 4 stretched the envelope and taught them to teach.
- Week 5 made sure leaders could say "yes" intelligently.

LAND IT

"Today is the week the envelope itself goes away. The skill is the same. The ceiling is much higher."

TRANSITION → "HERE'S HOW TODAY ACTUALLY RUNS."

How today runs

Deck → editor → deck. Repeated ten times.

The deck is the map

- Sets the stage for each module.
- Holds the architecture diagram we'll return to after each build.
- Marks every checkpoint and every break.

The editor is the ground

- For each build module I will leave the deck and we will work in the editor.
- You don't have to keep up live — checkpoints will catch you up.
- If the AI's output doesn't work in 3 minutes, paste the error back. Don't manually debug AI code.

Switch cues are red **Return cues are gold** **Speaker notes live on every slide** **2 breaks · 15 min each**

SPEAKER NOTES

SET EXPECTATIONS

The single most important thing you can do here is calibrate the room's expectation. They are about to watch you switch context constantly. Tell them why.

SAY VERBATIM

"You are going to see me leave this deck a lot. Every time we leave, we are building. Every time we come back, we will look at the same architecture diagram and color in another piece. If you fall behind in the editor, that is fine — we'll have a checkpoint version of the code at the end of every module."

REINFORCE THE 3-MINUTE RULE

Repeat this once now and at every debrief. It is the core operating discipline of the day.

TRANSITION → "SIX PRINCIPLES WE'LL LEAN ON ALL DAY."

Six principles for the day

Carry these into every prompt, every debrief, every deploy.

- **1. The conversation is the IDE.** The AI chat is your primary tool; the editor is for verification.
- **2. Scaffold → flesh out → integrate.** Skeleton first. Always.
- **3. The 3-minute rule.** Stuck for 3 minutes? Paste the error back to the AI.
- **4. Incremental deployment.** Deploy after every major feature, not at the end.
- **5. Interface-first design.** Define *what* before *how*.
- **6. Copy the error, not your frustration.** Give the AI the message, not the mood.

SPEAKER NOTES

TREAT AS A CREED

You'll come back to numbers 3 and 6 every time something breaks today. Plant them now.

STORY FOR PRINCIPLE 5

Briefly tell the room: "Saying 'write me a SQL query' constrains the AI. Saying 'I need a function that takes a student ID and returns their record' lets it pick the right tool. The interface comes first — the implementation is a detail."

TRANSITION → "HERE'S THE DAY AT A GLANCE."

The day at a glance

Ten modules. Two breaks. One deployed application.

0:00	**1. The Full-Stack Frontier** 30 MIN · FRAMING	0:30	**2. Environment Setup** 45 MIN · SETUP	1:15	**3. Backend from Scratch** 45 MIN · BUILD
2:00	**4. Data Layer** 45 MIN · BUILD	2:45	**— BREAK —** 15 MIN	3:00	**5. Frontend from Scratch** 45 MIN · BUILD
3:45	**6. Pages & Navigation** 45 MIN · BUILD	4:30	**7. AI Chat Integration** 45 MIN · BUILD	5:15	**— BREAK —** 15 MIN
5:30	**8. Auth & Middleware** 45 MIN · BUILD	6:15	**9. External Integrations** 45 MIN · BUILD	7:00	**10. Docker & Deployment** 45 MIN · BUILD
7:45	**Assessment & Wrap-Up** 15 MIN · CLOSE				

SPEAKER NOTES

PACING

Don't read this list. Glance at it, then summarize: "First two modules are framing and setup. Modules 3 through 10 are eight build cycles. Last fifteen minutes are assessment and a program close."

PACING ESCAPE VALVE

Tell them now: "If we run long, the module I will compress is Module 9, External Integrations — it's the most independent. Everyone will still ship a deployed container today."

TRANSITION → "QUICK LOOK AT WHAT WE'RE GOING TO BUILD."

What you walk out with today

A working full-stack app, modeled on Heywood TBS — built in one day, by you, with AI.

8.4k
BACKEND LOC
Go HTTP server, router, handlers, store interface, middleware.

4.4k
FRONTEND LOC
React + Vite + TypeScript, multi-page, role-aware UI.

35
API ENDPOINTS
Reference target — your version will start with ~5.

1
DOCKER CONTAINER
Backend + built frontend + data, runs anywhere.

Reference: Heywood TBS — built in days, by one person, working with AI. No team, no six-month timeline, no $2M budget.

SPEAKER NOTES

DEMO CUE

If Heywood is reachable on your machine, switch to it now and walk through Dashboard → Chat ("morning brief") → Settings → role picker. Two minutes total.

LAND THE REVEAL

After the demo, return to this slide and say verbatim: "8,400 lines of Go. 4,400 lines of React. 35 endpoints. Microsoft Graph. CAC. FIPS 140-3. Built in days, by one person, working with AI. Today you build the seed of that. Same skill, smaller scope."

WHAT YOU ACTUALLY SHIP

Set realistic expectations: "Your version today will have ~5 endpoints, three or four pages, a chat that calls one tool, role switching, and a container. That's the right scope for one day."

TRANSITION → SECTION DIVIDER FOR MODULE 1.

MODULE

01 /10

The Full-Stack Frontier

30 MIN **FRAMING · NO CODE**

A short mental-model shift from "I build Power Apps" to "I direct AI to build whatever the problem needs." We anchor the architecture we'll grow all day.

SPEAKER NOTES

PACE

Total 30 minutes. Three slides plus the architecture anchor — keep it tight.

TRANSITION → "WHY FULL-STACK? HERE'S WHERE POWER PLATFORM STOPS."

Where Power Platform stops

Most Marines never need to leave the low-code envelope. Some problems do.

LIMITATION	POWER PLATFORM	FULL-STACK
Custom AI chat with tool use	Not really possible	**Build exactly what you need**
Role-based dashboards	Power BI RLS, complex	**One app, four views**
Offline / disconnected ops	Needs internet + M365	**Container runs anywhere**
Data sovereignty	Lives in Microsoft cloud	**Lives where you put it**
Cost per user	$20+/user/mo	**$0 — open source, self-hosted**
Deployment flexibility	Microsoft cloud only	**Azure, AWS, on-prem, ship, field**

SPEAKER NOTES

BE HONEST ABOUT POWER PLATFORM

Don't trash it. Open with: "Power Platform is the right answer for most problems most of the time. The point of today isn't 'low-code is bad' — it's 'here's what to do when low-code can't reach the problem.'"

CONCRETE EXAMPLES

- Custom AI chat → "Copilot Studio is a great chatbot host but it can't run arbitrary tool calls against your private data the way we will today."
- Offline → "Imagine doing this on a ship without comms. Power Apps stops. A container does not."

TRANSITION → "HERE IS THE ARCHITECTURE WE ARE GOING TO GROW ALL DAY."

The four layers — empty for now

We will return to this exact diagram after every build module and color in another layer.

L1 · FRONTEND	**React Frontend**	What the user sees and clicks. Sends requests to the backend.
L2 · BACKEND	**Go HTTP Server**	Receives requests, processes data, returns JSON.
L3 · DATA	**Data Layer**	Where the data lives. JSON now, SQLite or Postgres later.
L4 · EXTERNAL	**External Services**	OpenAI for chat, Microsoft Graph for Outlook, etc.

SPEAKER NOTES

WHY THIS SLIDE MATTERS

This is the most important visual in the deck. It is the spine. Tell the room: "Mark this slide. We will see it eight more times today, and each time another box will light up."

WALK THE LAYERS

- L1 Frontend: "browser, what the user clicks."
- L2 Backend: "your server, the brain."
- L3 Data: "where state lives."
- L4 External: "anything you call out to — AI, Graph, weather, whatever."

TRANSITION → "HOW AI CHANGES WHO CAN BUILD THIS."

You will not learn to code today

You will learn to direct AI to write code, verify it works, and keep going when it doesn't.

What you actually need

- **What to ask for** — requirements (you learned this in Courses 1–4).
- **How to verify** — run it, check the browser, read the response.
- **How to debug** — paste errors back to the AI.
- **When to move on** — the 3-Minute Rule.

What changes vs. Power Platform

- The conversation is your IDE.
- The error message is your debugger.
- The interface is your design tool.
- The container is your deployment.

SPEAKER NOTES

REASSURANCE

The room will be intimidated. Say it out loud: "You felt the same way before Course 2 when you built your first Power App. By the end of today, you will have a deployed web application. Same process. Higher ceiling."

MODULE 1 COMPLETE CHECK

- Can articulate the four layers.
- Understands they will not "learn to code" — they will direct AI to code.
- Has the architecture diagram in their head.

TRANSITION → "NOW WE EARN THE RIGHT TO START. MODULE 2 — ENVIRONMENT SETUP."

MODULE

02 /10

Environment Setup

45 MIN

AI-GUIDED INSTALL

The most boring module and the most critical. If the environment isn't right, nothing else works. Today, we let the AI walk every student through their own machine.

SPEAKER NOTES

PLAN

Two slides: required tools and the "ask the AI to install them" prompt. Then we go to the editor and you actually do it. Expect 30–50% of the room to need help.

TRANSITION → "FIVE TOOLS. THE AI INSTALLS THEM FOR YOU."

Five tools, then we go

Don't memorize the install steps. Hand the list to your AI and let it walk you through your OS.

- **Go 1.22+** — backend language. go.dev/dl
- **Node.js 20+** — frontend build. nodejs.org
- **Git** — version control. git-scm.com
- **VS Code** — editor. code.visualstudio.com
- **Docker Desktop** — container runtime (Module 10). docker.com

STUDENT PROMPT — USE AS-IS

I need to set up a development environment on [Windows / Mac]. Install Go 1.22+, Node.js 20 LTS, Git, VS Code, and Docker Desktop. Give me step-by-step instructions for my OS, including how to verify each install. After installation I should be able to run `go version`, `node --version`, `npm --version`, `git --version` and see numbers for all four.

SPEAKER NOTES

THE TEACHING MOMENT

This is the first lesson in AI-assisted development: **let the AI install your tools.** Marines try to memorize commands; show them they don't have to.

COMMON GOTCHAS (CALL OUT BEFORE GOING TO EDITOR)

- Go not on PATH → restart terminal.
- `npm create vite` fails → `npm install -g npm@latest`.
- Docker requires admin → defer to Module 10 if needed.

TRANSITION → SWITCH-TO-EDITOR CUE IS NEXT. TAKE A BREATH BEFORE YOU SWAP WINDOWS.

▶ SWITCH TO THE EDITOR

→

Install & scaffold the project

Walk through: paste the install prompt, verify the four versions, then create the project skeleton: `my-staff-app` with `cmd/server/`, `internal/`, `web/` (Vite + React + TypeScript), and `data/`.

~30 min in editor — circulate while students install

SPEAKER NOTES

LIVE WORK

Share the terminal. Run the prompt. Talk through what the AI is doing. Then run the four verification commands.

SCAFFOLD COMMANDS

- `mkdir my-staff-app && cd my-staff-app`
- `go mod init my-staff-app`
- `mkdir -p cmd/server internal data`
- `npm create vite@latest web -- --template react-ts`
- `cd web && npm install && cd ..`

BACK TO DECK → WHEN EVERY STUDENT HAS ALL FOUR VERSIONS PRINTING AND THE PROJECT DIRECTORY EXISTS, RETURN.

Module 2 complete — runway built

The stack is still empty. Tomorrow it grows.

L1 · FRONTEND	**React Frontend**	Vite + React + TypeScript scaffolded in `web/`.
L2 · BACKEND	**Go HTTP Server**	Module initialised. Empty `cmd/server/` waiting for code.
L3 · DATA	**Data Layer**	Empty `data/` directory. No store yet.
L4 · EXTERNAL	**External Services**	No external services wired up.

- ✓ Go, Node, Git, VS Code installed and verified.
- ✓ Project directory exists with `go.mod` and `web/package.json`.
- ✓ Students used AI to install — not the docs.

Diagram status: **0 of 4 layers built.** Empty runway. We light the first lamp in Module 3.

SPEAKER NOTES

ANCHOR THE RHYTHM

This is the first time you return to the architecture diagram. Stop. Point at it. Say: "We will see this exact picture eight more times today. After Module 3, the backend box will be solid red. After Module 5, the frontend box. By the end of Module 10, every box is solid and we have shipped a container."

PACING

If you are running long here, skip the celebration and move straight to Module 3. Setup is supposed to be the dullest moment of the day.

TRANSITION → "MODULE 3 — FIRST SERVER. REAL CODE."

MODULE · BUILD

03 /10

Backend from Scratch

BUILD MODULE

45 MIN

First real server-side code — or rather, first time directing AI to write it. By the end, every student has a Go HTTP server returning JSON to a curl request.

SPEAKER NOTES

PACE

~45 min. Five slides plus heavy editor time. Plan: framing → editor → debrief → architecture → checkpoint.

TRANSITION → "HERE'S WHAT WE'RE BUILDING AND WHY."

Your first HTTP server

Stand up a Go server with one health endpoint and one items endpoint. Get to JSON in the browser.

▶ WHAT WE'RE ADDING

- A Go HTTP server on port 8080.
- A `/api/v1/health` endpoint that returns `{"status":"ok"}`.
- A separate `internal/api/router.go` with helpers.
- A `/api/v1/items` endpoint with hardcoded JSON.

Why this fits the architecture

This is layer L2 — the Go HTTP Server. Today it serves a hardcoded array; in Module 4 we'll wire it to a real data store; in Module 5 the React frontend will call it.

Principle in play: Scaffold → flesh out → integrate. We start with a server that does almost nothing. We make sure it works. *Then* we add features.

SPEAKER NOTES

POSTURE

Start naming the principles aloud as you frame each build. Today you say "scaffold → flesh out → integrate." Tomorrow students say it back to you.

TONE

Make the goal small. "We're trying to get five characters — `{"status":"ok"}` — into a browser. That is the whole goal of this module. Everything else is bonus."

TRANSITION → SWITCH-TO-EDITOR CUE IS NEXT.

▶ SWITCH TO THE EDITOR

→

Build the first server

Three prompts in sequence: stand up the server, extract a router, add the items endpoint. Run each, hit it with curl or the browser, paste any error back to the AI before debugging.

~30 min in editor

SPEAKER NOTES

DEMO FLOW

- Prompt 1 — single-file server with health endpoint and `-port` / `-dev` flags.
- Run `go run ./cmd/server -dev`. Hit `/api/v1/health` in the browser. Celebrate.
- Prompt 2 — extract `SetupRouter` into `internal/api/router.go` with `writeJSON` / `writeError` helpers.
- Prompt 3 — add hardcoded `/api/v1/items` with five military-flavored items.

COMMON ERRORS TO EXPECT (AND LET STUDENTS SEE YOU FIX)

- Port already in use → switch to 8081.
- Module-name mismatch → `go.mod` module name vs imports.
- Missing `go.sum` → `go mod tidy`.

3-MINUTE RULE IN ACTION

Demonstrate it. The first time you hit an error, count to three out loud, then paste it into the AI. Make the discipline visible.

BACK TO DECK → WHEN EVERY STUDENT SEES JSON IN THE BROWSER FOR BOTH ENDPOINTS, RETURN.

L2 lit — your first web server

Five characters of JSON. Zero lines of code you wrote yourself. Welcome to AI-assisted backend development.

L1 · FRONTEND	**React Frontend**	Scaffolded but not wired. Coming in Module 5.
L2 · BACKEND	**Go HTTP Server** ✓	Server, router, helpers, two endpoints. JSON over HTTP.
L3 · DATA	**Data Layer**	Hardcoded array for now. Real store in Module 4.
L4 · EXTERNAL	**External Services**	Nothing external yet.

- ✓ `go run ./cmd/server -dev` runs cleanly.
- ✓ Browser hits return real JSON on both endpoints.
- ✓ Code is split: `cmd/server/main.go` + `internal/api/router.go`.
- ✓ Each student ran the prompt → verify → iterate cycle at least twice.

Architecture status: **1 of 4 layers built (L2).** The first lamp is on.

SPEAKER NOTES

MARK THE MOMENT

This is the first emotional checkpoint of the day. Marines who five hours ago thought they couldn't write code now have a server they can curl. Stop and acknowledge it. Twenty seconds of silence is fine.

VOCABULARY CHECK

Ask the room: "What does the backend do, in one sentence?" Cold-call. Expected answer: "Receives a request, returns JSON." Anything close to that is right.

TRANSITION → "HARDCODED DATA IS FAKE. MODULE 4 MAKES IT REAL."

MODULE · BUILD

04 /10

Data Layer

BUILD MODULE 45 MIN

The most important conceptual move of the day: interface-first design. Define *what* the data layer does before you decide *how* it stores anything.

SPEAKER NOTES

THE POINT OF THIS MODULE

If they take one architecture lesson out of today, it's this one. Interfaces are the lever that lets a domain expert direct an AI without becoming a software engineer.

TRANSITION → "HERE'S WHY INTERFACE-FIRST MATTERS."

Interface first. Implementation later.

Define the contract — *list, get, create, update* — before you pick a backend.

▶ WHAT WE'RE ADDING

- A `DataStore` Go interface in `internal/data/store.go`.
- An `Item` struct with proper JSON tags.
- A `JSONStore` implementation reading `data/items.json`.
- The items endpoint rewired to read from the store.

Why interfaces are the lever

The Heywood DataStore has 27 methods and 5 backends — JSON, SQLite, Postgres, Excel, Hybrid. The rest of the application doesn't know which one is active. Define the interface today and you can:

- Start with JSON files (no setup).
- Upgrade to SQLite later (no other code changes).
- Switch to Postgres for prod (config change).

SPEAKER NOTES

LEAN ON PRINCIPLE 5

Principle 5 from the opening: **interface-first design**. Tell the room: "This is where you actually feel that principle pay off. We are not asking the AI to write a database. We are asking it to satisfy a contract. The same contract will be satisfied later by SQLite without us touching the rest of the app."

VOCABULARY

Check the room knows what an interface is. Define it in one sentence and write it on the whiteboard if you have one: **"A contract that says any data store must support these operations."**

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Define the DataStore interface

Three prompts: define `DataStore` + `Item` + `JSONStore`; generate 20 realistic seed items in `data/items.json`; rewire `main.go` to load the store and serve from it. Add `/api/v1/items/{id}`.

~30 min in editor

SPEAKER NOTES

LIVE WORK

- Have students examine the generated `store.go` together. Read the interface aloud.
- Use the AI to seed 20 items with military-flavored content. Look at the JSON file together.
- Curl `/api/v1/items` and `/api/v1/items/3`. Show that the data is now flowing through the store.

WATCH FOR

Marines copying interface methods into the JSON store and forgetting to satisfy them. Let the compiler error guide them — and let the AI fix it.

BACK TO DECK → WHEN EVERY STUDENT RETURNS 20 ITEMS AND A SINGLE ITEM, RETURN.

L3 lit — real data, behind a contract

The frontend doesn't exist yet, but the data path is real and the interface is the door.

L1 · FRONTEND	**React Frontend**	Scaffolded. Wires up in Module 5.
L2 · BACKEND	**Go HTTP Server** ✓	Reads from the store now, not hardcoded data.
L3 · DATA	**Data Layer** ✓	`DataStore` interface + `JSONStore` + 20 seed items.
L4 · EXTERNAL	**External Services**	Nothing external yet.

- ✓ Interface defined with 4+ methods.
- ✓ `JSONStore` reads from `data/items.json`.
- ✓ API serves real data; `/items` and `/items/{id}` both work.
- ✓ Each student can define "interface" in one sentence.

Architecture status: **2 of 4 layers built (L2 + L3).**

SPEAKER NOTES

QUICK RECAP

Pause and ask the room to recite: "Interface first. Implementation later." Get them to say it back. This is the most reused concept of the day.

TRANSITION → "BREAK TIME. FIFTEEN MINUTES. BE BACK AT [TIME]."

BREAK · BE BACK AT H+3:00

15 min

Stretch. Drink water. Don't open Slack. The next stretch is the longest of the day.

SPEAKER NOTES

HOLD THE BREAK

Hold a hard 15. Resist the urge to start early. Marines who fell behind in setup may use this time to catch up — circulate quietly.

PRE-FLIGHT BEFORE MODULE 5

- Confirm everyone's backend still runs after the break (terminal didn't get killed).
- Have the next prompt loaded so you can paste it the moment you're back.

RETURN → MODULE 5 — FRONTEND.

MODULE · BUILD

05 /10

Frontend from Scratch

BUILD MODULE 45 MIN

Stand up React + Vite + TypeScript, configure a proxy to the Go backend, and render the items table in a styled UI. The first end-to-end full-stack moment of the day.

SPEAKER NOTES

ENERGY CHECK

Coming back from break — re-anchor briefly. Two sentences: "Backend is live, data is real. Now we make humans able to see it."

TRANSITION → "FRONTEND FRAMING."

First end-to-end view

React fetches your Go API and renders a styled table. JSON file → Go server → React → browser.

▶ WHAT WE'RE ADDING

- Tailwind CSS configured in `web/`.
- Vite proxy: `/api` → `localhost:8080`.
- API client at `web/src/lib/api.ts`.
- `ItemsPage.tsx` — table with status / priority color-coding, loading + error states.

Why this fits the architecture

This module connects L1 to L2 — the frontend pulls data from your server through the proxy. Two terminals running side by side. From the user's point of view, this is the first time the application "exists."

Principle in play: Scaffold first. We are not styling. We are not adding features. We are getting *any* data on screen.

SPEAKER NOTES

WATCH THE TEMPTATION

Some students will want to make the table beautiful. Resist this. The job today is to get data on the page; we add features in Module 6.

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Wire the frontend to the backend

Two terminals: backend in one, `npm run dev` in the other. Configure Tailwind, set up the Vite proxy, build the API client, and render the items table at `localhost:5173`.

~30 min in editor

SPEAKER NOTES

DEMO FLOW

- Show two terminals side by side. Make the proxy concept visible.
- Common error to demo: CORS error → fix is the proxy, not CORS middleware.
- Show the network tab. Trace one request from React → Vite proxy → Go.

IF A STUDENT IS STUCK

Most failures here are PostCSS / Tailwind config or a missing proxy block in `vite.config.ts`. Both are 30-second AI fixes once they paste the error.

BACK TO DECK → WHEN EVERY STUDENT SEES A STYLED TABLE AT `LOCALHOST:5173`, RETURN.

L1 lit — full stack, end to end

JSON file → Go server → React → browser. Three of four layers are now real.

L1 · FRONTEND	**React Frontend** ✓	Items table at `localhost:5173`, color-coded.
L2 · BACKEND	**Go HTTP Server** ✓	Serving JSON to the proxied frontend.
L3 · DATA	**Data Layer** ✓	JSON store backing the items endpoint.
L4 · EXTERNAL	**External Services**	Coming online in Module 7 (OpenAI).

- ✓ React app loads at `localhost:5173`.
- ✓ Items table fetches and renders Go data.
- ✓ Status + priority columns color-coded.
- ✓ No console errors in dev tools.

Architecture status: **3 of 4 layers built.** The application now exists from a user's point of view.

SPEAKER NOTES

MARK THE MOMENT — SECOND VICTORY

This is bigger than Module 3. The data is now visible to a human. Stop. Have one student share their screen briefly so the rest of the room can see somebody else's work, not just yours.

TRANSITION → "ONE PAGE IS NOT AN APP. MODULE 6 MAKES IT ONE."

MODULE · BUILD

06 /10

Pages & Navigation

BUILD MODULE

45 MIN

Sidebar layout, client-side routing, dashboard with stats, item detail page. Turn the prototype into something that looks and feels like an application.

SPEAKER NOTES

PACE

This is a UI module. Visible progress is fast and motivating, but easy to over-style. Keep students focused on structure, not pixels.

TRANSITION → "FRAMING."

From one page to a real application

Layout, routes, dashboard, detail page. The shape of every internal tool you'll ever build.

▶ WHAT WE'RE ADDING

- `react-router-dom` with Layout, Dashboard, Items, Settings.
- Dark sidebar (navy `#1a1f36`) with nav highlighting.
- Item detail page at `/items/:id`.
- Dashboard stat cards, recent items, quick actions.

Why this fits the architecture

L1 is now a real frontend, not a single page. We're not adding new layers — we're filling out the one we just lit.

Principle in play: Iterative refinement. The first prompt gets you 70% there; the next two prompts get you to 95%.

SPEAKER NOTES

SET THE CONSTRAINT

Tell the room: "We are using Tailwind defaults. We are not picking fonts. We are not designing icons. The point is shape, not paint."

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

Build the multi-page UI

Three prompts in sequence: routing + sidebar layout, item detail page, dashboard with stats and recent items. Use Tailwind defaults. Active nav highlighting. Make titles in the items table click through to detail.

~30 min in editor

SPEAKER NOTES

LIVE WORK

- Demo navigating between pages — show URL changes without page reload.
- When the active link doesn't highlight, paste the issue back to the AI; this is the textbook iterative refinement loop.

WATCH FOR

404s on routes that look like they should work — usually missing route definition or wrong path. Let the AI fix.

BACK TO DECK → WHEN EVERY STUDENT HAS A WORKING SIDEBAR, DASHBOARD, ITEM LIST WITH CLICK-THROUGH, AND DETAIL PAGE.

L1 fleshed out — the prototype is gone

Same diagram. The frontend layer is now a real multi-page app, not a one-page demo.

L1 · FRONTEND	**React Frontend** ✓	Sidebar, routing, dashboard, items, detail. Looks like an app.
L2 · BACKEND	**Go HTTP Server** ✓	Same endpoints, more callers.
L3 · DATA	**Data Layer** ✓	JSON store still backing everything.
L4 · EXTERNAL	**External Services**	Next module — OpenAI.

- ✓ Sidebar nav works without page reload.
- ✓ Dashboard shows stats from API data.
- ✓ Detail page loads from items list.
- ✓ Active page highlighted in sidebar.

Architecture status: **3 of 4 layers built — L1 fully fleshed out.**

SPEAKER NOTES

SET UP MODULE 7

Module 7 is the most exciting module of the day for most rooms — it's when the AI starts calling their code. Tee it up: "Right now your app is a database with a face. In the next 45 minutes it grows a brain that can answer questions about itself."

TRANSITION → "MODULE 7 — AI CHAT WITH TOOL USE."

MODULE · BUILD

07 /10

AI Chat Integration

BUILD MODULE · 45 MIN

A chat backend that calls OpenAI, a tool the AI can call back into your data store, and a chat page on the frontend. The brain inside the app.

SPEAKER NOTES

WHY THIS IS THE HIGH POINT OF THE DAY

This is the module that makes the whole curriculum worth it. The AI calls **your** code and answers questions about **your** data — no hallucination, real lookups.

TRANSITION → "WHAT WE'RE BUILDING."

Tool use — the AI calls your code

When a user asks "how many high-priority items do I have?", the AI doesn't guess. It calls your `lookup_items` tool and reports the real count.

▶ WHAT WE'RE ADDING

- Chat service in `internal/ai/chat.go` calling `gpt-4o`.
- A `lookup_items` tool with status / priority / query params.
- `POST /api/v1/chat` endpoint.
- `ChatPage.tsx` with markdown rendering and loading states.

What is tool use?

Function calling lets the AI invoke functions in your application. The AI sees a list of tools you've offered, decides which to call, and uses the result in its answer.

This is what makes Heywood real. It does not hallucinate about student data. It queries the database and reports facts.

SPEAKER NOTES

MAKE IT CONCRETE

Walk the room through the round trip: user types question → frontend POSTs to `/chat` → Go calls OpenAI with the tool description → OpenAI says "call lookup_items with priority=high" → Go runs the lookup against the DataStore → Go sends the result back to OpenAI → OpenAI writes a sentence using the real numbers → Go returns the sentence to the frontend.

PRE-FLIGHT

Confirm everyone has `OPENAI_API_KEY` set in their shell. If not, do that first — this is the most common stuck-point.

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Build the chat with tool use

Three prompts: chat service against OpenAI; `lookup_items` tool definition; chat page with markdown rendering. End: ask "what are my high-priority items?" and watch the AI call your code.

~30 min in editor

SPEAKER NOTES

THE MOMENT TO LAND

Watch the server logs while you ask the question in the chat. Make sure students see the tool call printed in the terminal. That is the moment the abstraction becomes real.

COMMON STUCK-POINTS

- API key not set → `export OPENAI_API_KEY=…`.
- CORS on `/chat` → add to existing middleware.
- Tool response format wrong → paste the OpenAI error to the AI; it will fix its own tool definition.

BACK TO DECK → WHEN EVERY STUDENT GETS A REAL, DATA-GROUNDED ANSWER FROM THE CHAT.

L4 lit — the brain comes online

All four layers built. Your application can now answer questions about itself, grounded in real data.

L1 · FRONTEND	**React Frontend** ✓	+ ChatPage with markdown rendering.
L2 · BACKEND	**Go HTTP Server** ✓	+ `/chat` endpoint and chat service.
L3 · DATA	**Data Layer** ✓	Now consumed by humans *and* the AI tool.
L4 · EXTERNAL	**External Services** ✓	OpenAI `gpt-4o` with tool use.

- ✓ Chat sends and receives messages.
- ✓ AI uses tool calling to query the DataStore.
- ✓ Responses contain real data, not hallucinated items.
- ✓ Markdown renders correctly.

Architecture status: **4 of 4 layers built.** Every layer is real. The rest of the day is hardening.

SPEAKER NOTES

THIS IS THE APEX

Stop and let it land. This is the moment a Power Platform user crosses the line into something Power Platform cannot do. Acknowledge it: "Right now you are doing something Copilot Studio cannot do. You have built a chat that runs your code against your data inside your app. That is the threshold."

SET UP THE SECOND HALF

The remaining three modules harden the app: auth, an external integration, and a deployable container. Tell the room: "From here, every module makes the application more real-world."

TRANSITION → "BREAK TWO. FIFTEEN MINUTES."

BREAK · BE BACK AT H+5:30

15 min

Halfway through the back half. Stretch, water, kill any zombie servers in your terminal.

SPEAKER NOTES

TELL THEM

"After the break we have three modules. Two are short and one is the deploy. We will end on time."

PRE-FLIGHT BEFORE MODULE 8

- Confirm chat still works after the break.
- Have the auth middleware prompt loaded.

RETURN → MODULE 8 — AUTH.

MODULE · BUILD

08 /10

Auth & Middleware

BUILD MODULE

45 MIN

Different users see different data. Middleware is the gate guard that runs before every request reaches your handler. Today we build the role-aware version.

SPEAKER NOTES

FRAME THE METAPHOR

Use the gate guard image early. It's the easiest way for a Marine audience to internalize middleware.

TRANSITION → "FRAMING."

Middleware = gate guard

Code that runs before every request — checking credentials before anyone gets into the building.

▶ WHAT WE'RE ADDING

- Middleware package: CORS, security headers, auth, chain.
- `/auth/me` + `/auth/switch` endpoints.
- Role picker dropdown in the header (Admin / Staff / User).
- Role-based filtering — Admin sees all, User sees only their items.

Why this fits the architecture

No new layers. We're hardening L2 (the backend) and L1 (the frontend). The same data store now serves different users different views.

Principle in play: Scaffold, then layer. Today we use a cookie. In a real deployment this becomes CAC, OIDC, or whatever the unit standard is.

SPEAKER NOTES

SET THE REALISM DIAL

This is demo auth, not production auth. Say so out loud. "We are using a role cookie because it's the smallest thing that demonstrates the pattern. In production this is CAC + OIDC + a real session store. The pattern is the same."

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Add auth, middleware, role switcher

Three prompts: middleware package; auth endpoints + frontend role picker; role-based data filtering across the existing endpoints.

~30 min in editor

SPEAKER NOTES

DEMO FLOW

- Switch role to "User" — show fewer items.
- Switch role to "Admin" — show everything.
- Open dev tools → Application → Cookies, show the role cookie.
- Inspect the request headers — show the middleware setting role on the context.

COMMON STUCK-POINTS

- Cookie not sticking → check `SameSite` and credentials in fetch.
- Settings link still visible to Staff/User → conditional render in sidebar.

BACK TO DECK → WHEN EVERY STUDENT CAN SWITCH ROLES AND SEE DIFFERENT DATA.

All four layers — now role-aware

Same diagram. The application now treats different humans differently.

L1 · FRONTEND	**React Frontend** ✓	Role picker; conditional sidebar.
L2 · BACKEND	**Go HTTP Server** ✓	Middleware chain — CORS, security headers, auth.
L3 · DATA	**Data Layer** ✓	Filtered per-role at the handler.
L4 · EXTERNAL	**External Services** ✓	Same OpenAI integration.

- ✓ CORS, security headers, auth middleware all chained.
- ✓ Role picker persists across reload.
- ✓ Data is filtered by role.
- ✓ Students can define middleware in one sentence: "code that runs before every request."

Architecture status: **4 of 4 layers — hardened with auth.**

SPEAKER NOTES

QUICK CHECK

Cold-call: "What does middleware do?" Looking for: "Runs before every request — checks something, sets something, then lets the request continue."

TRANSITION → "MODULE 9 — YOUR APPLICATION MEETS THE OUTSIDE WORLD."

MODULE · BUILD

09 /10

External Integrations

BUILD MODULE 45 MIN

Choose your path. Path A — replace JSON with SQLite (no external accounts). Path B — connect to Outlook via Microsoft Graph (Azure AD required). Both prove the integration pattern.

SPEAKER NOTES

PACING VALVE

This is the module to compress if you're behind schedule. Tell the room up front: "Path A is the default. Path B is for Marines who already have Azure AD access. Either path counts."

TRANSITION → "PICK A PATH."

Choose your path

The interface from Module 4 means you can swap the implementation without breaking anything else.

Path A — SQLite backend

Default for most students. No external account needed.

- New `DataStore` implementation using `modernc.org/sqlite` (pure Go).
- Auto-create tables on startup.
- Parameterized queries.
- `-db` flag toggles JSON vs SQLite.

Path B — Microsoft Graph

For students with Azure AD access. Live Outlook data.

- OAuth2 client credentials flow.
- Token caching.
- Commercial & GCC High endpoints.
- New endpoints: `/calendar/today` and `/mail/summary`.

SPEAKER NOTES

BE OPINIONATED

Recommend Path A out loud. "If you don't have Azure AD app registration credentials in front of you right now, do Path A. Path B can wait until next week — and the SQLite skill is broadly more useful."

WHY PATH A IS THE TEXTBOOK PAYOFF

This is where Module 4's interface earns its keep. Tell the room: "We are about to swap the engine. Nothing else in the application changes. That is what interface-first design **buys you**."

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Wire your chosen integration

Path A: build the SQLite store, run with `-db sqlite`, restart, confirm data persists. Path B: build the Graph client, fetch today's calendar.

~30 min in editor

SPEAKER NOTES

LIVE WORK — PATH A

- Show the new SQLite store satisfying the existing interface.
- Restart with `-db sqlite`. Add an item via curl. Restart again. Show the item is still there.
- Switch back to `-db json` to show the upgrade is reversible.

LIVE WORK — PATH B

- Demo the OAuth2 token request flow once.
- Make sure your Tenant ID, Client ID, and Client Secret are in env vars before class — never hardcode.

BACK TO DECK → WHEN EVERY STUDENT HAS AT LEAST ONE EXTERNAL INTEGRATION RESPONDING WITH REAL DATA.

L3 or L4 deepened — interfaces paid off

Whichever path you took, the rest of the application didn't change. That is the interface paying its rent.

L1 · FRONTEND	**React Frontend** ✓	Unchanged.
L2 · BACKEND	**Go HTTP Server** ✓	Unchanged.
L3 · DATA	**Data Layer** ✓	Path A: now SQLite-backed; data persists across restart.
L4 · EXTERNAL	**External Services** ✓	Path B: OpenAI + Microsoft Graph (Outlook).

- ✓ App connected to at least one external service.
- ✓ Integration works end-to-end.
- ✓ Rest of the app didn't need to change — interfaces did the work.

Architecture status: **4 of 4 — deepened.** Time to ship.

SPEAKER NOTES

CONNECT THE DOTS

Spell it out: "Six modules ago we wrote an interface. We just swapped the implementation behind it and nothing else in the app moved. **That** is the lever. Remember it for everything you build after today."

TRANSITION → "FINAL BUILD MODULE — PACKAGE AND SHIP."

MODULE · BUILD · FINAL

10 /10

Docker & Deployment

BUILD MODULE 45 MIN

A multi-stage Dockerfile bundles backend, frontend, and data into one container. It runs identically on a laptop, a server, Azure, a ship, or a field TOC.

SPEAKER NOTES

FRAME THE STAKES

"This is the third victory of the day. Right now your app exists on your laptop. After this module, it exists everywhere — same bytes, any machine."

TRANSITION → "FRAMING."

One container. Runs anywhere.

Multi-stage build — Node bakes the frontend, Go compiles the binary, Alpine runs the result.

▶ WHAT WE'RE ADDING

- Multi-stage `Dockerfile`: web-builder, go-builder, runtime.
- Go server serves the React build in production mode.
- SPA fallback for non-API routes.
- `docker build` + `docker run` locally on port 8080.
- *Optional:* push to ACR and deploy to Azure Container Apps.

Why this fits the architecture

The container collapses L1, L2, and L3 into a single artifact. L4 (OpenAI, Graph) stays external — you pass keys via environment variables.

Principle in play: Incremental deployment. We are not waiting until "the end" to ship. The container *is* the end.

SPEAKER NOTES

WHITEBOARD THE THREE STAGES

Sketch the multi-stage build out loud: stage 1 builds the React bundle, stage 2 compiles the Go binary, stage 3 is a tiny Alpine image with just the binary, the bundle, and the data folder.

TRANSITION → SWITCH-TO-EDITOR.

▶ SWITCH TO THE EDITOR

→

Build & run the container

Generate the multi-stage Dockerfile. `docker build -t my-staff-app .`. `docker run -p 8080:8080 -e OPENAI_API_KEY=…`. Open `localhost:8080` — full app from a single container. Optional: deploy to Azure.

~30 min in editor

SPEAKER NOTES

DEMO DISCIPLINE

- Show the image size — talk about why multi-stage builds matter.
- Run the container, then stop your local dev servers — show the app still works because everything is now in the container.

COMMON STUCK-POINTS

- No `.dockerignore` → image bloated by `node_modules`.
- SPA fallback missing → 404 on direct navigation to non-root pages.
- Azure CLI not logged in → `az login` first.

BACK TO DECK → WHEN EVERY STUDENT HAS THE APP RUNNING FROM THE CONTAINER ON `LOCALHOST:8080`.

All four layers, packaged and shipped

The application now exists outside your laptop. Same bytes, any machine, any environment.

L1 · FRONTEND	**React Frontend** ✓	Built bundle, served by Go in prod mode.
L2 · BACKEND	**Go HTTP Server** ✓	Single binary. Serves the API *and* the SPA.
L3 · DATA	**Data Layer** ✓	Bundled into the image (or mounted as a volume).
L4 · EXTERNAL	**External Services** ✓	Wired via environment variables — OpenAI, Graph, etc.

- ✓ Image builds successfully.
- ✓ Container runs and serves the full application.
- ✓ Students understand: *one container = entire application.*
- ✓ *(Bonus)* Deployed to Azure with a public URL.

Architecture status: **4 of 4 layers — packaged.** The application now lives anywhere.

SPEAKER NOTES

LAND THE MOMENT

Pause for ten seconds. Look at the diagram together. Say it out loud: "Eight hours ago this diagram was empty. Now every layer is solid and the whole thing fits in one container that you can run anywhere. You did this. AI wrote the code. You directed the work."

TRANSITION → "LAST FIFTEEN MINUTES — ASSESSMENT, THEN WE CLOSE THE PROGRAM."

Stuck? Compare against the answer key

The finished reference for today's build lives in this repo at `heywood-inventory/`. Use it *after* the 3-Minute Rule fires twice on the same prompt — never before.

How to use it: open the file for the module you're stuck in, read it top to bottom, then re-prompt your AI with the gap. Don't copy — close the reference once you see the missing piece. Full guidance in `heywood-inventory/docs/STUDENT_GUIDE.md`.

MODULE	WHAT YOU BUILT	REFERENCE FILE(S)
3 — Backend	Server + router	`cmd/server/main.go` · `internal/api/router.go`
4 — Data Layer	Interface + JSON store + seed	`internal/data/store.go` · `internal/data/json_store.go` · `data/items.json`
5 — Frontend	Vite proxy, API client, items table	`web/vite.config.ts` · `web/src/api/client.ts` · `web/src/pages/Items.tsx`
6 — Pages & Nav	Sidebar, dashboard, detail	`web/src/App.tsx` · `web/src/layouts/Layout.tsx` · `web/src/pages/Dashboard.tsx` · `web/src/pages/ItemDetail.tsx`
7 — AI Chat	Chat service + tool + chat page	`internal/ai/chat.go` · `web/src/pages/ChatPage.tsx`
8 — Auth & Middleware	Middleware, role picker, role filtering	`internal/middleware/middleware.go` · `web/src/components/RolePicker.tsx` · `internal/api/handlers.go`
9 — Integrations	SQLite (Path A) or Graph (Path B)	`internal/data/sqlite_store.go` · `internal/integrations/graph.go`

SPEAKER NOTES

WHY THIS SLIDE EXISTS

Until today the reference was an instructor-only artifact and stuck students had to flag you down. Now they can self-unblock without dragging the room. Make sure every student knows the path: `heywood-inventory/` in the course repo, file table in `docs/STUDENT_GUIDE.md`.

THE DISCIPLINE

- **3-Minute Rule fires once** → paste the error into the AI chat. **Don't** open the reference yet.
- **3-Minute Rule fires twice on the same prompt** → open the matching reference file, read it top to bottom, then re-prompt. Close the file once the gap is obvious.
- **Module “done” but architecture diagram doesn't match** → walk the corresponding reference file, spot the missing piece, prompt the AI to add it.
- **Finished early** → pick one feature from the reference (extra endpoint, role, polish) and add it to your build with prompts.

REMINDER — TARGETS, NOT THE REFERENCE

The reference has 13 endpoints, 5 pages, both Path A and Path B. Students are aiming for the seed target on slide T (~5 endpoints, 3–4 pages, one tool, one container). Don't let the reference become the target.

TRANSITION → "LAST FIFTEEN MINUTES — ASSESSMENT, THEN WE CLOSE THE PROGRAM."

CLOSING

A /Assess

Assessment & Wrap-Up

15 MIN

SELF-ASSESSMENT + HANDOFF

Confirm what you shipped against the rubric. Update your frontier map. Submit for the certificate. Then we close the six-week program.

SPEAKER NOTES

RHYTHM

Three slides for assessment, then we move into the program close. Keep this brisk so the close has time to land.

TRANSITION → "RUBRIC."

What “done” looks like

Minimum gets you certified. Target is the bar to set for your real-world build.

CRITERION	MINIMUM (CERTIFIED)	TARGET
API endpoints	3 returning JSON	**5+ with full CRUD**
Frontend pages	2 with navigation	**4+ with sidebar, routes, detail**
Data layer	Reads from JSON	**Interface-based, swappable**
AI integration	Chat sends/receives	**Tool use against live data**
Authentication	Role picker present	**Role-based filtering**
Deployment	Container runs locally	**Deployed to Azure with a public URL**

SPEAKER NOTES

HOW TO USE THIS SLIDE

Have students self-assess against the minimum column right now. If they hit every cell on the left, they pass. If they hit cells on the right, they have something portfolio-worthy.

BE GENEROUS ON MINIMUMS

The point of today was the rep, not the polish. If a Marine has a container running on their laptop with chat working against their data, that is a pass.

TRANSITION → "HOW TO SUBMIT, FRONTIER MAP, AND WHAT COMES NEXT."

Submit & certify

Finish the student-page checklist, then generate your certificate from the EDD site.

Three steps

1. Open the student version of this course and check off every box on the completion checklist.
2. Take the knowledge check at the bottom of the page.
3. Go to *Generate Certificate*, pick *Full-Stack AI-Assisted Development*, fill in your name / rank / unit, and print or save as PDF.

Where to find it

- **Student page:** */courses/student/fullstack.html*
- **Progress dashboard:** */courses/progress.html*
- **Certificate:** */courses/certificate.html*

Your progress is saved in your browser. To carry it across machines, use *Export Training Record* on the progress page.

SPEAKER NOTES

HANDOFF TO THE CERTIFICATE FLOW

Walk the room through the three pages briefly — student page, progress, certificate. The certificate generator will only show this course in the dropdown once every box on the completion checklist is checked. Make sure students know that.

QUICK REMINDER

Storage is per-browser. If they took earlier courses on a different machine, they'll only see today's certificate available. That's expected.

TRANSITION → "UPDATE YOUR FRONTIER MAP."

Move your frontier

Update your map from Course 1. The boundary just shifted.

Add to your map

- **Can now do:** direct AI to build & deploy a full-stack web app.
- **Still learning:** complex DB design, prod security hardening, CI/CD.
- **Next frontier:** what problem at your unit needs a custom application that Power Platform can't solve?

What comes next (recommended)

- **Week 1–2 after class:** rebuild today's app from scratch, no notes. Muscle memory.
- **Week 3–4:** pick a real unit problem. Use the Problem Definition Worksheet (Course 3).
- **Week 5–8:** build it. Same prompt → verify → iterate cycle. Plan 80–120 hours.
- **Week 9+:** deploy, gather feedback, iterate. Document with the Documentation Package.

For comparison: a contractor would quote 6–12 months and $500K+ for the same application.

SPEAKER NOTES

TIME EXPECTATIONS

Be honest about the curve. First real full-stack app: 80–120 hours. Second: 40–60. Third: 20–30. The skill compounds because they get better at **asking** — not at typing.

TRANSITION → "NOW WE CLOSE THE SIX-WEEK PROGRAM."

PROGRAM · CAPSTONE CLOSE

VI /SixWeeks

Closing the Six-Week Program

~7 MIN

ALL HANDS

A short, honest close. Where you started. What you built across the cohort. What you owe forward. Where to keep going.

SPEAKER NOTES

POSTURE SHIFT

Slow down here. The next four slides are the program close. Speak more deliberately. Make eye contact (or its Teams equivalent — pause and watch reactions).

TRANSITION → "LOOK AT THE JOURNEY."

Where you started, where you stand

The arc, in one slide.

W1 **AI Fluency Fundamentals**
YOU LEARNED THE MANAGEMENT FRAMING. THE 80% QUIT PROBLEM.

W2 **Builder Orientation**
YOUR FIRST PROTOTYPE. DECOMPOSITION UNDER PRESSURE.

W3 **Platform Training**
THREE DEPLOYED POWER PLATFORM TOOLS. CENTAUR AND CYBORG.

W4 **Advanced Workshop**
YOU MAPPED YOUR FRONTIER. YOU TAUGHT SOMEONE ELSE.

W5 **Supervisor Orientation**
LEADERS LEARNED TO EVALUATE PROPOSALS AND PROTECT THE APPRENTICE PIPELINE.

W6 **Full-Stack (today)**
YOU SHIPPED A CONTAINERIZED APPLICATION BUILT END-TO-END WITH AI.

SPEAKER NOTES

READ IT SLOW

Don't speed-read. Pause on each line. The point is not information — they lived it. The point is collective acknowledgement.

IF YOU HAVE COHORT NUMBERS

If you have data on how many tools the cohort built across all six weeks, name it: "Across this cohort, we shipped **N** Power Platform tools and **M** containers." Specifics make this slide land harder than abstractions.

TRANSITION → "WHAT YOU OWE FORWARD."

What you owe forward

The pipeline only stays open if you choose to hold it open.

> **“That training pipeline that was always implicit has broken, and it has to be reconstructed.”**
>
> — MOLLICK, ON THE APPRENTICESHIP CRISIS (COURSE 1)

Three commitments

- **Make a junior Marine review your AI output** before you ship it. They learn judgment by reading good work and bad.
- **Share your failure cases.** Where AI degrades your work is as valuable as where it accelerates it.
- **Teach the next cohort.** Co-teach a course. Run a section meeting. Write up one tool.

The line you do not cross

- Don't cut a junior role because AI handled the routine work.
- Architects who never laid a brick build buildings that fall down.
- Your bench in five years is the room you protect today.

SPEAKER NOTES

POSTURE

This is the most serious slide in the deck. Slow down. Mean it.

TIE BACK TO COURSE 5

If supervisors are in the room, name the connection: "This is the same point Course 5 makes to leaders. It's the same point you make to yourselves now that you can build at this speed."

TRANSITION → "WHERE TO KEEP GOING."

Where to keep going

Bookmark these. Use them in the order that fits the next problem you pick.

On the EDD site

- **SOP** — the operating doctrine for AI-assisted builds.
- **Toolkit** — interactive worksheets and checklists.
- **Templates** — Problem Definition, Documentation Package, prompt patterns.
- **Resources** — readings, references, links to the underlying research.

Practice

- Rebuild today's app from scratch. No prompts. No notes.
- Find a real unit problem. Define it. Build it.
- Co-teach one course. Anyone in this room is qualified to start.
- Push your work to the EDD GitHub. Other units learn from it.

SPEAKER NOTES

TACTICAL CLOSE

Tell the room exactly what to do in the next 24 hours: "Tonight, generate your certificate. This week, pick one of these four practice steps. Don't pick all four. Don't pick none."

TRANSITION → FINAL SLIDE. CAPSTONE CLOSE.

COURSE 6 · CAPSTONE COMPLETE

You wrote the requirements. AI wrote the code. You shipped the container.

That is not “learning to code.” That is Expert-Driven Development at the highest level — domain experts building exactly what they need, at the speed of conversation. Now go build something your unit actually needs.

EXPERT-DRIVEN DEVELOPMENT · WEEK 6 OF 6 · MCD-MONTEREY

SPEAKER NOTES

LAND IT

Read the headline aloud, slowly. Pause for three full seconds before you say "Now go build something your unit actually needs." Then thank the room and stop talking.

WHAT TO DO AFTER CLASS

- Post the recording (if any) to the cohort channel.
- Drop the certificate page link in the channel.
- Schedule a 30-minute office hour 1 week out for the rebuild attempt.
- Identify the two strongest students in the room as candidates for instructor certification on this course.

END → STOP SHARING. DONE.